Demanda, Provisionamento e Desempenho

Provisionamento e Operações de Infraestrutura, Atividade 2

Grupo: Eurico Pereira, Pedro Veloso e Victor Carneiro

Neste trabalho desenvolvemos três aplicações backend, cada uma limitada por um recurso diferente da máquina: CPU, memória e disco. As três foram implantadas numa VM do VirtualBox e testadas sob carga em cinco configurações de vCPU e RAM, com cinco níveis de demanda em cada uma. A ideia era observar em que ponto o desempenho começa a cair e se dar mais recurso para a VM resolve o problema. Como o professor pediu, fizemos o experimento completo para as três aplicações, e não só para uma.

Todo o código, os scripts e os resultados brutos estão no repositório do grupo. Os gráficos deste relatório foram gerados a partir de `results/_analysis/`.

1. Aplicações desenvolvidas

As três aplicações usam Python com FastAPI, servidas pelo Gunicorn com workers Uvicorn. O código de cada uma é dividido em controllers (HTTP), services (regras) e repositories (acesso a dados). Os testes automatizados sobem um servidor de verdade numa porta livre e fazem requisições reais, sem mocks. Os gargalos foram escolhidos de propósito, e cada app foi configurada para que o gargalo dela apareça com clareza.

1.1 Cofre Digital (CPU)

O Cofre Digital simula operações de criptografia feitas inteiramente em Python puro, sem numpy. Ele tem três rotas de trabalho:

- `GET /primos/{n}`: gera todos os primos até n por divisão por tentativa;
- `POST /assinar`: aplica SHA-256 repetidas vezes sobre um texto (de 50 mil a 500 mil rodadas);
- `GET /cifrar-matriz/{tamanho}`: multiplica duas matrizes com um laço triplo em Python.

Como o código é Python puro, cada processo fica preso ao GIL e usa no máximo um núcleo. Por isso o Gunicorn sobe um worker por vCPU (`WORKERS=auto`). Assim, o desempenho deveria crescer com o número de vCPUs e não depender da RAM. No teste de carga, cada usuário virtual escolhe entre primos, assinatura e matriz na proporção 3:3:2.

1.2 Galeria de Filtros (memória)

A Galeria de Filtros gera imagens sintéticas e as guarda num cache LRU em memória. Sobre essas imagens ela aplica filtros (negativo e tons de cinza) e monta mosaicos. As rotas principais são:

- `POST /imagens/{id}`: gera uma imagem de 1000 a 3000 pixels de lado (3 a 27 MB) e a coloca no cache;
- `GET /imagens/{id}/filtro`: aplica um filtro a uma imagem do cache, criando uma cópia nova. Se a imagem não está no cache, responde 404 (miss);
- `GET /mosaico`: junta as imagens mais recentes numa só;
- `GET /cache/stats` e `DELETE /cache`: estatísticas e limpeza do cache.

O cache ocupa no máximo 40% da RAM da VM, e a app roda com um único worker para que o cache seja um só. A carga do k6 usa 128 imagens diferentes, que juntas somam cerca de 1,7 GB. Esse conjunto não cabe no cache em nenhum cenário, então sempre há descarte de imagens, e com pouca RAM a VM passa a usar swap. A mistura de operações é criar 3, filtrar 6, mosaico 1 e estatísticas 1.

Na calibração percebemos que a galeria gastava CPU demais e, com 3 GB, quem limitava era a CPU, não a memória. Reduzimos esse custo de duas formas. O ruído das imagens passou a ser sorteado num bloco de 512×512 e repetido pela imagem, o que derrubou o tempo para gerar uma imagem de 50 para 12 ms. O filtro de cinza passou a usar contas com inteiros, caindo de 24 para 9 ms. Também aumentamos de 48 para 128 o número de imagens diferentes, porque antes todas cabiam no cache.

1.3 Bolsa Fictícia (disco)

A Bolsa Fictícia é uma corretora simulada. Investidores consultam preços e carteiras e enviam ordens de compra e venda de quatro ações. Cada ordem é gravada num banco SQLite em modo WAL com `synchronous=FULL` e também num arquivo de livro-caixa, com `fsync` a cada escrita. Isso força o sistema a esperar o disco confirmar cada gravação, e o disco vira o gargalo. A app roda com 4 workers.

No teste, cada usuário virtual cria um investidor e depois escolhe ações com estes pesos: operar com a estratégia automática 10, enviar ordem 2, mexer nos preços 2, ver carteira 3, listar ações 3 e listar ordens 2. A rota `/livro-caixa/stats` informa o tempo médio de `fsync` observado, e `POST /admin/reset` zera o banco entre um nível e outro.

1.4 Como a aplicação e a infraestrutura foram monitoradas

O monitoramento foi montado para atrapalhar o mínimo possível a VM:

- **Métricas da VM:** dentro da VM roda só o `node_exporter`, que ocupa uns 15 MB de RAM. O Prometheus, no notebook, coleta as métricas dele a cada 1 segundo: CPU (user, system, iowait), RAM, swap, disco e rede. O Grafana, também no notebook, mostra tudo ao vivo durante o teste.
- **Métricas da aplicação:** o k6 registra latência (média, p50, p95), vazão em requisições por segundo e falhas. Ao fim de cada nível, o script também salva as estatísticas da própria app: acertos e descartes do cache na galeria e tempo de `fsync` na bolsa.
- **Exportação:** ao terminar cada app, o script `metricas_vm.py` consulta o Prometheus na janela exata do teste e grava um CSV com uma linha por segundo. O `analyze.py` junta os CSVs e os resumos do k6, descarta a rampa inicial de cada nível e gera a tabela `resumo.csv` e os gráficos.

Um detalhe importante para ler os gráficos: a "CPU da VM" é calculada como 100% menos o tempo ocioso, então inclui o tempo em que a CPU fica parada esperando o disco (iowait). Isso pesa principalmente na bolsa, como mostramos na seção 3.

2. Configuração da VM e deploy das aplicações

Usamos o disco disponibilizado na disciplina (`ubuntu_serve_2204_poi.vdi`, Ubuntu Server 22.04 com Python 3.10). A configuração da VM foi feita à mão, pela interface do VirtualBox, seguindo o roteiro do professor.

2.1 Criação da VM

1. Copiamos o arquivo `.vdi` e criamos uma VM do tipo Linux / Ubuntu 64-bit usando esse disco.
2. Em Configurações, Rede, Adaptador 1 (NAT), Avançado, Redirecionamento de portas, criamos as regras da tabela abaixo. Todas usam TCP, IP do hospedeiro `127.0.0.1` e IP do convidado em branco.
3. Ligamos a VM e entramos com o usuário `osboxes` (senha `osboxes.org`).

Regra	Porta no notebook	Porta na VM	Uso
ssh	2222	22	o script de teste entra na VM
cofre	8001	8001	Cofre Digital
galeria	8002	8002	Galeria de Filtros
bolsa	8003	8003	Bolsa Fictícia
node	9100	9100	métricas da VM (Prometheus)

Tabela 1. Redirecionamento de portas do NAT.

2.2 Deploy dentro da VM

Dentro da VM instalamos os pacotes necessários, clonamos o repositório e criamos um ambiente virtual Python para cada app:

```
sudo apt update
sudo apt install -y git python3-venv openssh-server prometheus-node-exporter
git clone <repositório> ~/POI-lab-01
cd ~/POI-lab-01/apps
for app in cofre_digital galeria_filtros bolsa_ficticia; do
  (cd $app && python3 -m venv .venv && .venv/bin/pip install -r requirements.txt)
done
```

Cada app sobe com o script `run.sh`, que chama o Gunicorn com as variáveis de ambiente da app (porta, número de workers, tamanho do cache etc.). Para conferir, subimos uma app na VM e chamamos a rota `/health` a partir do notebook:

```
cd ~/POI-lab-01/apps/cofre_digital && WORKERS=auto ./run.sh     # na VM
curl localhost:8001/health                                      # no notebook
```

Os requisitos para o deploy são poucos: Python 3.10 ou mais novo, as dependências de cada `requirements.txt` (FastAPI, Gunicorn, Uvicorn, numpy, Pillow e psutil) e as portas liberadas no NAT. O `node_exporter` e o SSH sobem sozinhos no boot, então depois de ligar a VM não é preciso fazer login.

2.3 Preparação do notebook

No notebook copiamos a chave SSH para a VM (`ssh-copy-id -p 2222 osboxes@127.0.0.1`), para que o script entre sem pedir senha. Instalamos o k6 e subimos o Prometheus e o Grafana com `docker compose up -d` na pasta `monitoring/`. O Grafana já abre com um painel de CPU, RAM, disco e rede da VM.

2.4 Troca de cenário

Para mudar de cenário desligamos a VM com `sudo poweroff`, ajustamos a memória e o número de processadores em Configurações, Sistema, e ligamos de novo. Não dá para mudar vCPU e RAM de uma VM com estado salvo, por isso desligamos sempre de verdade.

3. Testes de stress

3.1 Cenários de provisionamento

Usamos cinco cenários. Eles foram escolhidos para permitir comparar o efeito de mudar um recurso de cada vez:

Cenário	vCPU	RAM	Para que serve
1cpu-1gb	1	1 GB	ponto de partida, o mais apertado
1cpu-2gb	1	2 GB	comparado com 1cpu-1gb, muda só a RAM
2cpu-1gb	2	1 GB	comparado com 1cpu-1gb, muda só a CPU
2cpu-2gb	2	2 GB	dobra os dois recursos
3cpu-3gb	3	3 GB	o cenário com mais recurso

Tabela 2. Cenários de provisionamento da VM.

3.2 Como os testes foram feitos

A carga foi gerada de fora da VM, pelo k6 rodando no notebook e acessando as apps pelas portas do NAT. Assim o gerador de carga não disputa CPU e RAM com a aplicação medida.

Cada cenário é rodado por um único comando, `loadtest/run_scenario.sh <cenário>`. Antes de começar, o script consulta o Prometheus para confirmar que a VM tem mesmo a quantidade de vCPU e RAM do nome do cenário. Depois, para cada uma das três apps, ele:

1. sobe a app na VM por SSH e espera a rota `/health` responder;
2. para cada nível de demanda, zera o estado da app (cache ou banco), roda o k6 com 10 s de rampa e 1 minuto com o número de usuários do nível, salva as estatísticas da app e espera 15 s;
3. para a app e exporta as métricas da VM daquela janela de tempo.

Cada usuário virtual faz uma requisição, espera entre 0,5 e 1,5 s e repete. Os níveis de demanda são os mesmos em todos os cenários, para que os gráficos possam ser comparados. Escolhemos os níveis com uma calibração rápida no cenário mais forte (`3cpu-3gb`), de forma que até esse cenário piore claramente no último nível:

Aplicação	Usuários simultâneos por nível
Cofre Digital	4, 8, 16, 48, 128
Galeria de Filtros	10, 20, 40, 120, 400
Bolsa Fictícia	25, 50, 100, 200, 500

Tabela 3. Níveis de demanda.

Cada cenário leva cerca de 21 minutos, e os cinco juntos, perto de 1h45. Rodamos uma vez cada nível. Durante os testes o notebook ficou na tomada e sem outros programas pesados abertos.

3.3 Resultados

Nos gráficos, o eixo horizontal é o número de usuários simultâneos e cada linha é um cenário. A degradação aparece quando a vazão para de subir enquanto os usuários aumentam e a latência p95 dispara. Para ver o efeito do provisionamento, basta comparar em que ponto isso acontece em cada cenário.

Cofre Digital

	1cpu-1gb	1cpu-2gb	2cpu-1gb	2cpu-2gb	3cpu-3gb
vazão máxima (req/s)	7,7	8,0	13,8	13,5	19,0
p95 com 16 usuários	2,1 s	2,1 s	0,9 s	1,1 s	0,6 s
p95 com 128 usuários	23,0 s	23,3 s	13,5 s	13,8 s	10,4 s
CPU com 16 usuários	99%	99%	77%	74%	56%

Tabela 4. Cofre Digital: principais números por cenário.

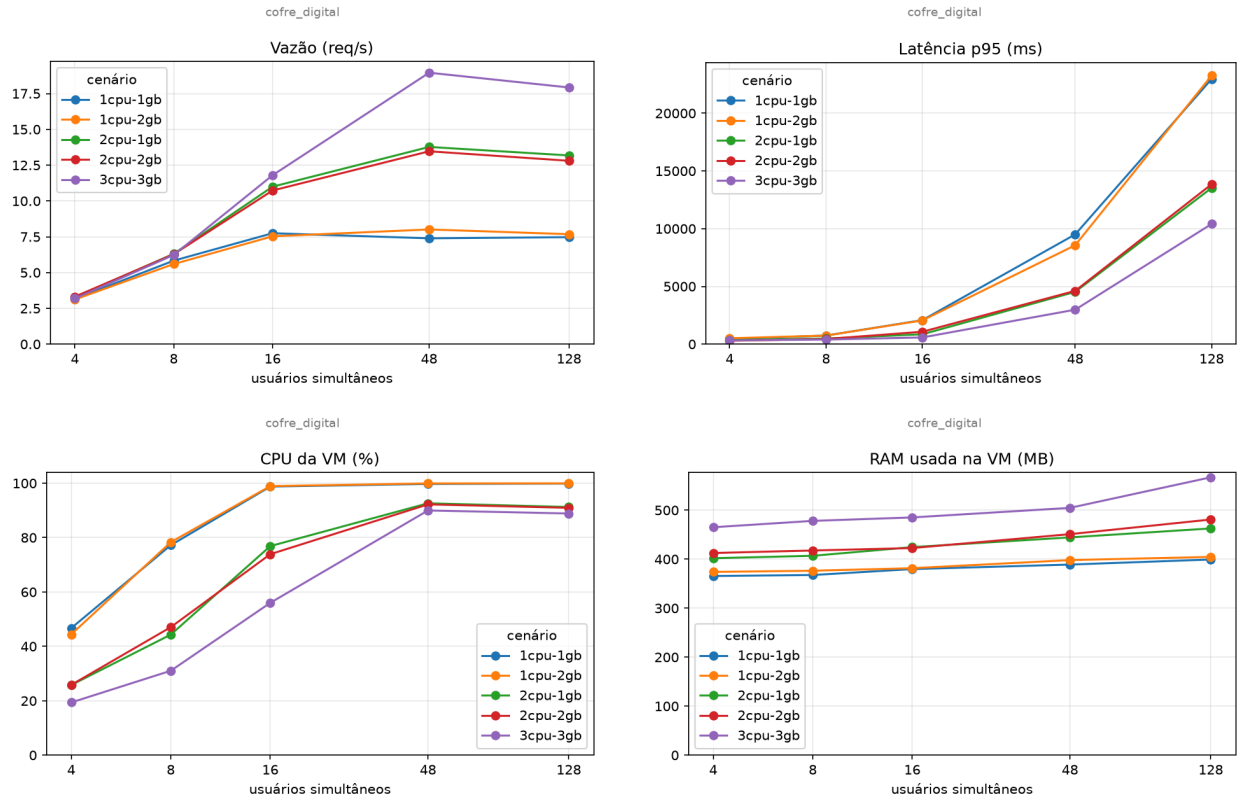


Figura 1. Cofre Digital: vazão, latência p95, CPU e RAM por número de usuários.

O Cofre se comportou como uma aplicação limitada por CPU. A vazão máxima cresce quase na mesma proporção das vCPUs: perto de 8 req/s com 1 vCPU, 13,5 com 2 e 19 com 3. A RAM não faz diferença. `1cpu-1gb` e `1cpu-2gb` dão praticamente o mesmo resultado, e o mesmo acontece com `2cpu-1gb` e `2cpu-2gb`. A app usa menos de 600 MB em qualquer cenário.

Com 1 vCPU, a CPU já chega a 99% com 16 usuários. A partir daí a vazão não sobe mais, e cada usuário a mais só aumenta a fila: com 128 usuários, o p95 passa de 23 s. Com 3 vCPUs, a saturação só aparece com 48 usuários, e o p95 com 128 cai para 10 s. Mais CPU empurra o ponto de saturação para a direita, mas uma demanda alta o bastante sempre degrada o serviço.

Galeria de Filtros

	1cpu-1gb	1cpu-2gb	2cpu-1gb	2cpu-2gb	3cpu-3gb
vazão máxima (req/s)	36	74	36	110	134
vazão com 400 usuários	20	60	12	110	134
p95 com 120 usuários	24,6 s	1,1 s	17,2 s	0,2 s	0,2 s
swap máximo	1,5 GB	420 MB	1,7 GB	310 MB	0
acertos no cache (120 usuários)	23%	47%	23%	48%	73%
falhas com 400 usuários	0,3%	0%	23%	0%	0%

Tabela 5. Galeria de Filtros: principais números por cenário.

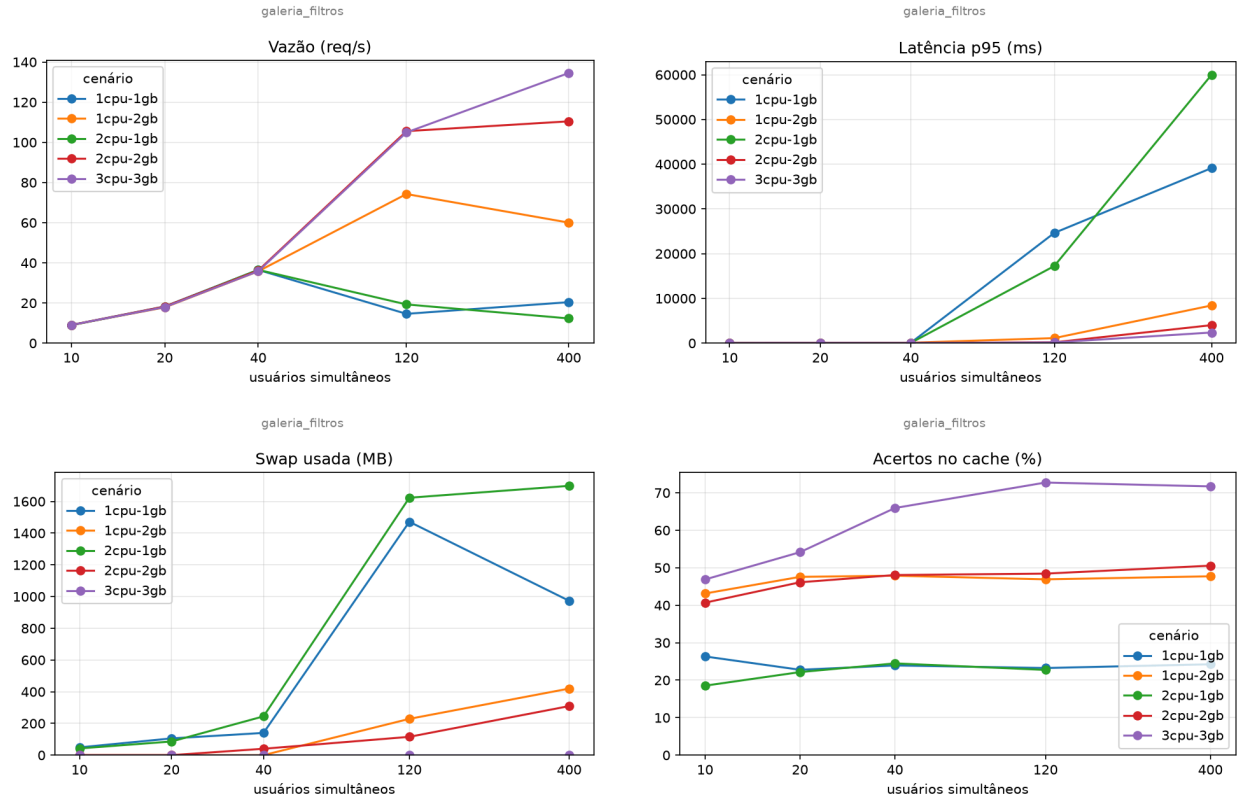


Figura 2. Galeria de Filtros: vazão, latência p95, swap e acertos no cache.

Na galeria o efeito da RAM é o mais claro de todo o experimento. Os dois cenários com 1 GB se comportam quase igual. Até 40 usuários tudo vai bem, mas com 120 a VM entra em swap pesado (1,5 a 1,7 GB), o disco fica mais de 90% ocupado só com a troca de páginas e a vazão despenca para menos da metade. Ter 2 vCPUs em vez de 1 não salvou o `2cpu-1gb`: com 400 usuários, 23% das requisições dele estouraram o timeout de 60 s do k6, e a app ficou tão travada que nem respondeu ao pedido de estatísticas no fim do nível.

Já dobrar só a RAM (`1cpu-1gb` para `1cpu-2gb`) dobrou a vazão máxima, de 36 para 74 req/s, e os acertos no cache subiram de 23% para 47%. Com 2 GB a memória deixa de ser o problema principal, e o limite passa a ser a CPU, que fica em 100% a partir de 120 usuários no `1cpu-2gb`. Com 3 GB o swap fica zerado e o cache acerta 73% das vezes. Mesmo assim, com 400 usuários o p95 chega a 2,4 s, porque a CPU volta a ser o limite. A galeria mostra bem que, quando um recurso deixa de faltar, o gargalo muda de lugar.

Bolsa Fictícia

	1cpu-1gb	1cpu-2gb	2cpu-1gb	2cpu-2gb	3cpu-3gb
vazão com 500 usuários (req/s)	86	97	94	104	101
p95 com 100 usuários	5,3 s	2,4 s	2,9 s	2,1 s	0,1 s
p95 com 500 usuários	14,4 s	11,5 s	12,5 s	10,9 s	12,4 s
disco ocupado (500 usuários)	97%	96%	93%	96%	92%
iowait (500 usuários)	69%	66%	41%	40%	26%
fsync médio (500 usuários)	13,7 ms	11,3 ms	9,7 ms	8,8 ms	7,9 ms

Tabela 6. Bolsa Fictícia: principais números por cenário.

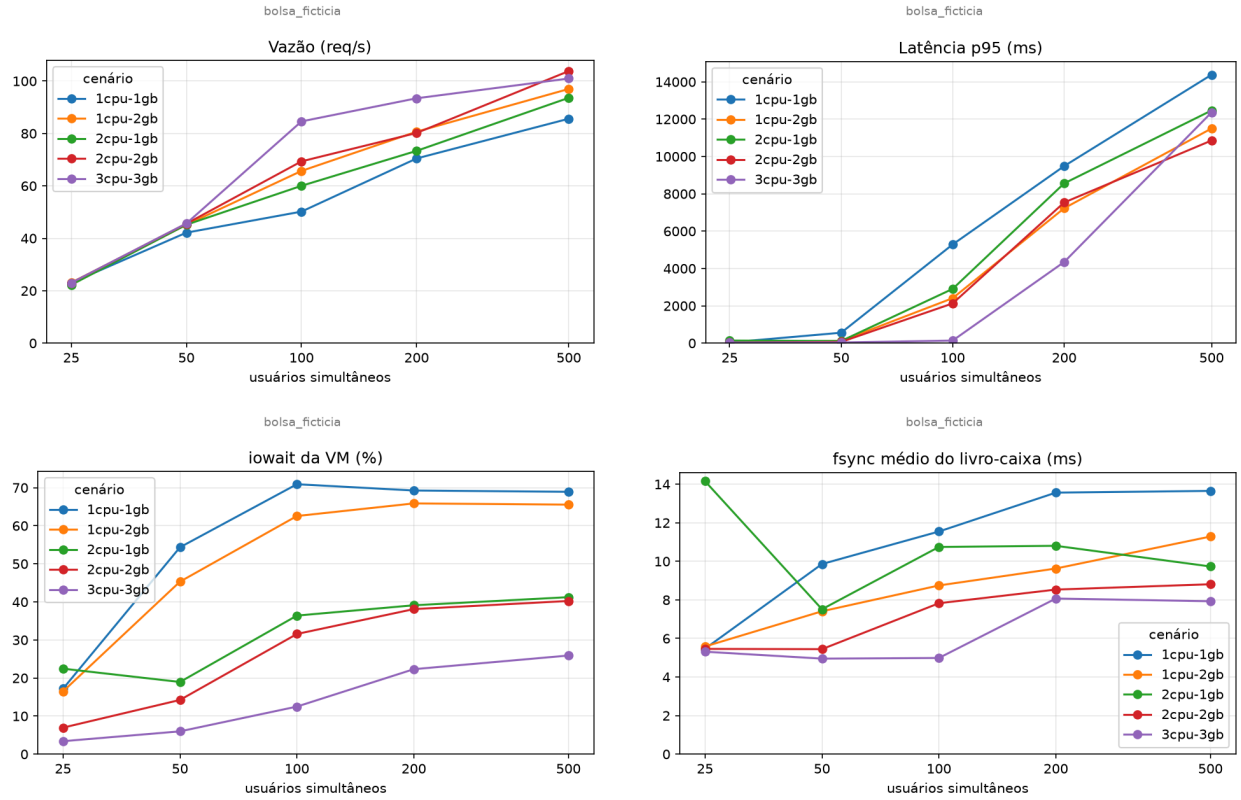


Figura 3. Bolsa Fictícia: vazão, latência p95, iowait e tempo médio de fsync.

A bolsa mostrou o comportamento de uma aplicação limitada por disco. A vazão máxima ficou entre 86 e 104 req/s em todos os cenários, mesmo triplicando CPU e RAM. Com 200 usuários ou mais, o disco fica ocupado mais de 90% do tempo e o `fsync` do livro-caixa sobe de 5 ms para 8 a 14 ms. A partir daí a latência explode: com 500 usuários, o p95 fica entre 11 e 14 s em todos os cenários, e aparecem algumas falhas (menos de 1%).

Mais CPU ajuda só um pouco. O iowait cai de 69% para 26% e o `3cpu-3gb` segura melhor a carga intermediária: com 100 usuários, o p95 dele é de 0,1 s, contra 2 a 5 s nos outros. Mas no topo todos os cenários batem no mesmo limite, que é a velocidade do disco. Nesse caso, só um disco mais rápido resolveria.

Aqui aparece a armadilha da métrica de CPU. No `1cpu-1gb` com 500 usuários, a CPU da VM aparece com 99%, mas só 23% disso é processamento de verdade (user e system). Os outros 69% são a CPU parada esperando o disco. Quem olhasse só para a CPU poderia achar que faltava processador, quando na verdade faltava disco.

3.4 Conclusões

Nas três aplicações, aumentar a demanda acabou degradando o desempenho em todos os cenários. O padrão se repete: até certo ponto a vazão cresce junto com os usuários e a latência fica baixa; depois o recurso que limita a app satura, a vazão para de subir e a latência dispara. O que muda de uma app para outra é qual recurso satura e, por isso, qual provisionamento ajuda.

- **Cofre Digital (CPU):** mais vCPUs aumentam a capacidade quase na mesma proporção. Mais RAM não muda nada.
- **Galeria de Filtros (memória):** com 1 GB a VM entra em swap e o serviço quase para, mesmo com mais CPU. Dobrar a RAM dobra a vazão. Com memória de sobra, o limite passa a ser a CPU.
- **Bolsa Fictícia (disco):** nem CPU nem RAM resolvem. Todos os cenários param perto de 100 req/s, porque o limite é o disco.

Na prática, isso mostra que antes de provisionar mais recurso é preciso descobrir qual é o gargalo. Dobrar a CPU da galeria com 1 GB, ou dar mais CPU para a bolsa, gasta recurso sem resolver o problema.

3.5 Limitações

- Rodamos uma vez cada nível, então diferenças pequenas entre cenários (como 97 contra 104 req/s na bolsa) podem ser só variação normal.
- O p95 de 60 s da galeria no `2cpu-1gb` com 400 usuários é o limite de timeout do k6. A latência real seria ainda maior.
- O disco da VM é um arquivo no SSD do notebook. Na calibração, com níveis de 15 s, a bolsa chegou a 166 req/s com 200 usuários. Com níveis de 1 minuto ficou entre 70 e 93. O disco virtual aguenta um pico curto de escritas melhor do que escrita contínua, e um disco físico diferente daria outros números.
- Todo o tráfego passa pelo NAT do VirtualBox. Usamos a mesma rede em todos os cenários, então a comparação entre eles continua justa.